

Practical Week-7-2520090046

Task-1

```
GNU nano 7.2 memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;

void code_function()
{
    printf("Code address      : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 200;
    int stack_var = 300;

    int *heap_var = (int *)malloc(sizeof(int));

    if (heap_var == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    *heap_var = 400;

    printf("=====\n");
    printf(" Linux Process Address Space\n");
    printf("=====\n");

    printf("Code address      : %p\n", (void *)code_function);
    printf("Global address    : %p\n", (void *)&global_var);
    printf("Static address    : %p\n", (void *)&static_var);
    printf("Heap address      : %p\n", (void *)heap_var);
    printf("Stack address     : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}

shivani@Shivani:~$ nano memory.c
shivani@Shivani:~$ ls
Desktop      command_resolver.c  filecopy.c      parser.c        process_state    signal_project    task1.c
Documents    command_resolver    fork_demo       pipe_exec      process_state.c  source.txt        task2.c
Downloads    destination.txt     fork_demo.c     pipe_exec.c    producer_consumer producer_consumer.c state_test.c    task3.c
memory       fifo_project        memory.c        process.c       sample.txt       state_test.c      task1
Music        filecopy            parser          process.c
Pictures     FIRST.c             filecopy.c
Public       FIRST.c.save
Templates    OSSP_2
Videos       OSSP1
shivani@Shivani:~$ gcc memory.c -o memory
shivani@Shivani:~$ ./memory
=====
Linux Process Address Space
=====
Code address      : 0x5614f6b7d61c
Global address    : 0x5614f6b7d910
Static address    : 0x5614f6b7d914
Heap address      : 0x5614f8c3e2a0
Stack address     : 0x7ffd3c9a682c
shivani@Shivani:~$
```

Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Observation:

The program demonstrates the addresses of code, global, static, heap, and stack variables and analyzes the Linux process address space, which consists mainly of text/code, data, BSS, heap, and stack regions.

TASK-2

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
GNU nano 7.2 memory.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main()
{
    static int static_var = 200;
    int stack_var = 300;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 400;

    printf("Process ID: %d\n", getpid());

    printf("Global variable address : %p\n", (void *)&global_var);
    printf("Static variable address : %p\n", (void *)&static_var);
    printf("Heap variable address   : %p\n", (void *)heap_var);
    printf("Stack variable address  : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");
    printf("Press Enter to terminate.\n");

    getchar();

    free(heap_var);

    return 0;
}
```

```

root@Shamikahasini:~# pgrep memory
666
root@Shamikahasini:~# cat /proc/666/maps
5a70b08fe000-5a70b08ff000 r--p 00000000 08:30 2072
ot/memory /ro
5a70b08ff000-5a70b0900000 r-xp 00001000 08:30 2072
ot/memory /ro
5a70b0900000-5a70b0901000 r--p 00002000 08:30 2072
ot/memory /ro
5a70b0901000-5a70b0902000 r--p 00002000 08:30 2072
ot/memory /ro
5a70b0902000-5a70b0903000 rw-p 00003000 08:30 2072
ot/memory /ro
5a70c2898000-5a70c28b9000 rw-p 00000000 00:00 0
ap] [he
791665000000-791665028000 r--p 00000000 08:30 11921
r/lib/x86_64-linux-gnu/libc.so.6 /us
791665028000-7916651b0000 r-xp 00028000 08:30 11921
r/lib/x86_64-linux-gnu/libc.so.6 /us
7916651b0000-7916651ff000 r--p 001b0000 08:30 11921
r/lib/x86_64-linux-gnu/libc.so.6 /us
7916651ff000-791665203000 r--p 001fe000 08:30 11921
r/lib/x86_64-linux-gnu/libc.so.6 /us
791665203000-791665205000 rw-p 00202000 08:30 11921
r/lib/x86_64-linux-gnu/libc.so.6 /us
791665205000-791665212000 rw-p 00000000 00:00 0
791665216000-791665219000 rw-p 00000000 00:00 0
79166521e000-791665220000 rw-p 00000000 00:00 0
791665220000-791665224000 r--p 00000000 00:00 0
ar] [vv
791665224000-791665226000 r--p 00000000 00:00 0
ar_vclock] [vv
791665226000-791665228000 r-xp 00000000 00:00 0
so] [vd
791665228000-791665229000 r--p 00000000 08:30 11915
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 /us
791665229000-791665254000 r-xp 00001000 08:30 11915
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 /us
791665254000-79166525e000 r--p 0002c000 08:30 11915
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 /us
79166525e000-791665260000 r--p 00036000 08:30 11915
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 /us
791665260000-791665262000 rw-p 00038000 08:30 11915
r/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 /us

```

```

root@Shamikahasini:~# nano memory.c
root@Shamikahasini:~# nano memory.c
root@Shamikahasini:~# gcc memory.c -o memory
root@Shamikahasini:~# ./memory
Process ID: 666
Global variable address : 0x5a70b0902010
Static variable address : 0x5a70b0902014
Heap variable address : 0x5a70c28982a0
Stack variable address : 0x7ffc32ec452c

Process is running...
Press Enter to terminate.

```

```

root@shivani:~# cat /proc/666/status | grep -E 'VmSize|VmRSS|VmData|VmStk|VmExe|VmLib'
VmSize:      2688 kB
VmRSS:       1540 kB
VmData:      224 kB
VmStk:       132 kB
VmExe:        4 kB
VmLib:      1748 kB
root@shivani:~#

```

```

666:  ./memory
Address      Kbytes      RSS      Dirty Mode Mapping
00005a70b08fe000      4         4         0 r---- memory
00005a70b08ff000      4         4         0 r-x-- memory
00005a70b0900000      4         4         0 r---- memory
00005a70b0901000      4         4         4 r---- memory
00005a70b0902000      4         4         4 rw--- memory
00005a70c2898000     132         4         4 rw--- [ anon ]
0000791665000000     160        156         0 r---- libc.so.6
0000791665028000    1568        992         0 r-x-- libc.so.6
00007916651b0000     316         64         0 r---- libc.so.6
00007916651ff000      16         16        16 r---- libc.so.6
0000791665203000       8          8         8 rw--- libc.so.6
0000791665205000      52         20        20 rw--- [ anon ]
0000791665216000      12          8         8 rw--- [ anon ]
000079166521e000       8          4         4 rw--- [ anon ]
0000791665220000      16          0         0 r---- [ anon ]
0000791665224000       8          0         0 r---- [ anon ]
0000791665226000       8          4         0 r-x-- [ anon ]
0000791665228000       4          4         0 r---- ld-linux-x86-64.so.2
0000791665229000     172        172         0 r-x-- ld-linux-x86-64.so.2
0000791665254000      40         40         0 r---- ld-linux-x86-64.so.2
000079166525e000       8          8         8 r---- ld-linux-x86-64.so.2
0000791665260000       8          8         8 rw--- ld-linux-x86-64.so.2
00007ffc32ea5000     132         12        12 rw--- [ stack ]
-----
total kB          2688      1540      96

```

Observation

The `/proc/666/maps` file was used to observe the process's virtual memory layout. The code, data, heap, stack, and shared library regions were identified with their virtual addresses and permissions.

